Réalité Augmentée
from scratch

Rapport de synthèse — Semestres 1, Semaines 1 à 4

Modules 01 à 04 · Bibliothèque mathématique `NkMath`

Règle absolue : Zéro bibliothèque tierce — C++17 + STL uniquement

Auteur du cours	Teuguia Tadjuidje Rodolf Sédéris · Rihen
Durée totale	3 Semestres · 162h · 21 Modules
Modules couverts	M01 – M04 (Semaines 1–4)

0 Introduction

Ce rapport résume les quatre premières semaines du cours *Réalité Augmentée from scratch*. L'objectif de ce semestre est de construire la bibliothèque mathématique `NkMath` intégralement en C++17, sans aucune dépendance externe. Chaque module pose une brique indispensable au pipeline AR complet : arithmétique flottante fiable, algèbre vectorielle, transformations matricielles et représentation des rotations par quaternions.

0 Contents

1 Semaine 1 — Module 01 : IEEE 754 & Arithmétique Flottante **2**
1.1 Pourquoi ce module est fondamental . . . 2
1.2 Représentation binaire IEEE 754 . . . 2
1.3 Valeurs spéciales . . . 2
1.4 Epsilon machine et ULP . . . 2
1.5 Cancellation catastrophique . . . 2
1.6 Sommation de Kahan . . . 3
1.7 Stratégie float/double dans NkMath . . . 3
1.8 Travaux pratiques . . . 3

2 Semaine 2 — Module 02 : Vecteurs — Vec2d, Vec3d, Vec4d **3**
2.1 Rôle des vecteurs dans le pipeline AR . . . 3
2.2 Vec2d . . . 3
2.3 Dot product — interprétation géométrique . . . 3
2.4 Vec3d et cross product . . . 4
2.5 Gram-Schmidt — base orthonormale . . . 4
2.6 Vec4d — coordonnées homogènes . . . 4
2.7 Travaux pratiques . . . 4

3 Semaine 3 — Module 03 : Matrices — Mat3d, Mat4d **4**
3.1 Convention column-major — critique pour OpenGL . . . 4
3.2 Mat4d — structure et produit . . . 4
3.3 Inverse par Gauss-Jordan avec pivot partiel . . . 5
3.4 Matrice de rotation — formule de Rodrigues . . . 5
3.5 Matrice View — LookAt . . . 5
3.6 TRS — Translation, Rotation, Scale . . . 5
3.7 Travaux pratiques . . . 5

4 Semaine 4 — Module 04 : Quaternions **5**
4.1 Problème du gimbal lock . . . 5
4.2 Définition et structure . . . 6
4.3 SLERP — interpolation sphérique . . . 6
4.4 Méthode de Shepperd — conversion Quat ↔ Mat3 . . . 6
4.5 Travaux pratiques . . . 6

1 Semaine 1 — Module 01 : IEEE 754 & Arithmétique Flottante

Durée : 6 heures

1.1 Pourquoi ce module est fondamental

Tout le pipeline AR manipule des flottants : coordonnées 3D, matrices de projection, résidus de calibration, valeurs de pixels. Une erreur d'arrondi invisible peut faire diverger des algorithmes itératifs (Levenberg-Marquardt), produire une homographie incorrecte, ou décaler une pose de plusieurs centimètres. Ce module est la fondation de tout code numérique fiable.

1.2 Représentation binaire IEEE 754

Un `float` 32 bits est découpé en trois champs :

Champ	Largeur	Rôle
Signe	1 bit	0 = positif, 1 = négatif
Exposant	8 bits	Valeur biaisée (biais = 127)
Mantisse	23 bits	Fraction avec bit implicite `1.`

$$\text{Valeur} = (-1)^{\text{signe}} \times 2^{\text{exposant}-127} \times 1.\text{mantisse}$$

Un `double` 64 bits porte 11 bits d'exposant et 52 bits de mantisse, soit environ 15–16 chiffres décimaux significatifs (contre 7–8 pour le `float`).

En C++17, la seule façon légale d'accéder aux bits d'un flottant est via `std::memcpy` vers un entier, afin d'éviter tout comportement indéfini lié au *strict aliasing*.

1.3 Valeurs spéciales

- **±Inf** : exposant `0xFF`, mantisse nulle (ex. `1.0f/0.0f`).
- **NaN** : exposant `0xFF`, mantisse non nulle ; *contagieux* — toute opération avec NaN produit NaN. La seule comparaison valide est `std::isnan(x)`, car `NaN != NaN` est toujours vrai.
- **Subnormaux** : exposant `0x00`, mantisse non nulle — représentation des très petits nombres au prix d'une perte de précision.
- **±0** : deux patterns de bits distincts, mais mathématiquement égaux.

1.4 Epsilon machine et ULP

Epsilon machine (ε) : plus petite valeur telle que $1.0 + \varepsilon \neq 1.0$. Pour `float`, $\varepsilon \approx 1.19 \times 10^{-7}$; pour `double`, $\varepsilon \approx 2.22 \times 10^{-16}$.

ULP (*Unit in the Last Place*) : espacement entre deux flottants consécutifs au voisinage d'une valeur donnée. L'ULP *varie* avec la magnitude : près de 1000, l'ULP est environ 1.22×10^{-4}, soit 10^3 fois plus grand que près de 1.

1.5 Cancellation catastrophique

La soustraction de deux flottants presque égaux détruit presque tous les bits significatifs du résultat. C'est l'erreur la plus dangereuse du pipeline car elle est silencieuse (pas de NaN, pas d'Inf). Exemple : calculer la variance par la formule naïve $\frac{\sum x_i^2 - (\sum x_i)^2/n}{n-1}$ peut produire une valeur *négative* à cause de l'annulation.

Remède — Algorithme de Welford : mise à jour incrémentale de la moyenne et de la variance qui n'effectue jamais de soustraction de grandes valeurs proches.

1.6 Sommation de Kahan

L'algorithme de Kahan compense les erreurs d'arrondi lors de la sommation de nombreux termes grâce à une variable de compensation qui capture les bits perdus à chaque addition. Sur un million de `0.1f`, `std::accumulate` donne une erreur de l'ordre de 11, tandis que Kahan donne un résultat correct à 6 chiffres significatifs.

1.7 Stratégie float/double dans NkMath

- Calculs internes (matrices, SVD) : **double** pour la précision de calibration.
- Upload GPU (`glVertexAttribPointer`, `glUniformMatrix4fv`) : **float**, car GLSL est optimisé pour le 32 bits.
- Types exportés : `Vec3d` (double) et `Vec3f` (float) selon le contexte.

1.8 Travaux pratiques

Les TP de cette semaine demandent : (1) implémenter `inspectFloat` pour visualiser les trois champs IEEE 754 de valeurs cibles ; (2) démontrer empiriquement les pertes de précision avec Kahan et Welford ; (3) créer le fichier `NkMath/Float.h` contenant toutes les fonctions utilitaires (`isFiniteValid`, `nearlyZero`, `approxEq`, `kahanSum`) avec au moins 30 tests unitaires.

2 Semaine 2 — Module 02 : Vecteurs — Vec2d, Vec3d, Vec4d

Durée : 9 heures

2.1 Rôle des vecteurs dans le pipeline AR

Les vecteurs sont la structure de données la plus utilisée du pipeline. Chaque position dans l'espace est un `Vec3d`, chaque pixel est un `Vec2d`, chaque coordonnée homogène est un `Vec4d`. Un tableau de `Vec3d` peut être passé directement à `glVertexAttribPointer` *sans copie* si le layout mémoire est correct, d'où l'usage de `static_assert` sur `sizeof` et `offsetof`.

2.2 Vec2d

Structure avec deux `double` (x, y) continus en mémoire. Fournit :

- Opérateurs arithmétiques : `+`, `-`, `*`, `/`, composés, unaire.
- Accès par index via `operator[]` (utilise le fait que les membres sont contigus).
- `Norm()`, `Normalized()`, `IsNormalized()`.
- **Dot product** : $a \cdot b = a_x b_x + a_y b_y = |a||b| \cos\theta$.
- **Cross 2D** (scalaire) : $a_x b_y - a_y b_x$ — signe indique si b est à gauche ou à droite de a.
- **Lerp** linéaire.

Garantie : `static_assert(sizeof(Vec2d)==16)`.

2.3 Dot product — interprétation géométrique

$$a \cdot b = |a||b| \cos\theta \begin{cases} = 0 & \Rightarrow \text{perpendiculaires} \\ > 0 & \Rightarrow \text{angle aigu} \\ < 0 & \Rightarrow \text{angle obtus} \end{cases}$$

Usages dans le pipeline : test face avant/arrière, éclairage de Phong, projection d'un point sur une droite, test d'appartenance à un plan.

2.4 Vec3d et cross product

Le **produit vectoriel** $a \times b$ produit un vecteur perpendiculaire à a et b, dont la norme est $|a||b|\sin\theta$ (aire du parallélogramme). Il respecte la règle de la main droite et est *non-commutatif* : $a \times b = -(b \times a)$.

Formule explicite :

$$(a \times b) = \begin{pmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{pmatrix}$$

Projection et rejection : décomposer un vecteur a en composante parallèle et perpendiculaire à b :

$$\text{proj}(a, b) = b\,\frac{a \cdot b}{|b|^2}, \qquad \text{reject}(a, b) = a - \text{proj}(a, b)$$

2.5 Gram-Schmidt — base orthonormale

À partir de trois vecteurs quelconques (non coplanaires), l'algorithme de Gram-Schmidt construit une base orthonormale $\{u, v, w\}$ en trois étapes : normaliser le premier vecteur, ôter sa composante du second puis normaliser, ôter les deux composantes du troisième puis normaliser. Utilisé pour construire la matrice *LookAt* et la décomposition polaire (Module 05).

2.6 Vec4d — coordonnées homogènes

Les coordonnées homogènes unifient points et directions :

- $w = 1$: **point** dans l'espace (la translation s'applique).
- $w = 0$: **direction** (la translation ne s'applique pas).

La déhomogénéisation se fait par `ToVec3()` : diviser (x, y, z) par w.

2.7 Travaux pratiques

Les TP demandent : (1) implémenter `Vec2d` complet avec 20 tests unitaires sur le dot product, le cross 2D, la normalisation et l'accès par index ; (2) implémenter `Vec3d` avec cross product et Gram-Schmidt, testés sur des triplets aléatoires ; (3) projeter les 8 coins d'un cube unitaire en 2D via une projection perspective simple et tracer les 12 arêtes dans une image PPM 512×512.

3 Semaine 3 — Module 03 : Matrices — Mat3d, Mat4d

Durée : 9 heures

3.1 Convention column-major — critique pour OpenGL

OpenGL stocke les matrices en **column-major** : dans un tableau linéaire, les éléments de la première colonne viennent en premier. L'accès C++ est `data[col*4 + row]`. Lors de l'upload GPU, `glUniformMatrix4fv` avec `GL_FALSE` suppose déjà du column-major. Une transposition accidentelle produit des transformations complètement incorrectes.

3.2 Mat4d — structure et produit

- Stockage : tableau `double data[16]` column-major.
- `operator()(row, col)` : accès intuitif en notation mathématique.
- Produit matriciel 4×4 : 64 multiplications et 48 additions.
- `operator*(Vec4d)` : transformation d'un vecteur homogène.

- `Transposed()`, `ToFloat(float[16])` pour l'upload GPU.

3.3 Inverse par Gauss-Jordan avec pivot partiel

L'inversion opère sur la matrice augmentée $[M \mid I]$:

1. Pour chaque colonne, trouver la ligne avec le plus grand coefficient en valeur absolue (**pivot partiel**) et l'amener en position de pivot — réduit les erreurs numériques.
2. Normaliser la ligne pivot.
3. Éliminer la colonne courante dans toutes les autres lignes.
4. Extraire la partie droite $I \to M^{-1}$.

La fonction retourne `false` si la matrice est singulière.

3.4 Matrice de rotation — formule de Rodrigues

Rotation d'angle θ autour d'un axe unitaire $\hat{n} = (n_x, n_y, n_z)$:

$$R = I\cos\theta + (1 - \cos\theta)\,\hat{n}\hat{n}^T + \sin\theta\,[\hat{n}]_\times$$

En pratique, chacun des 9 coefficients est calculé explicitement à partir de $c = \cos\theta$, $s = \sin\theta$, $t = 1 - c$.

3.5 Matrice View — LookAt

Construit la vue caméra à partir de trois vecteurs de l'espace monde :

1. $\hat{f} = \text{normalize}(\text{target} - \text{eye})$ — axe forward.
2. $\hat{r} = \text{normalize}(\hat{f} \times \text{worldUp})$ — axe right.
3. $\hat{u} = \hat{r} \times \hat{f}$ — axe up corrigé.

La matrice View combine la rotation $[\hat{r}, \hat{u}, -\hat{f}]$ et la translation $(-\hat{r}\cdot\text{eye},\ -\hat{u}\cdot\text{eye},\ \hat{f}\cdot\text{eye})$.

3.6 TRS — Translation, Rotation, Scale

$$M_{\text{TRS}} = T \times R \times S$$

L'ordre est critique : d'abord l'échelle (locale), puis la rotation, puis la translation monde. Inverser cet ordre produit des transformations incorrectes (ex. une mise à l'échelle après rotation déforme l'objet dans l'espace monde).

3.7 Travaux pratiques

Les TP demandent : (1) implémenter `Mat4d` avec produit et inversion, vérifier $M \times M^{-1} = I$; (2) construire un rasteriseur logiciel dans une image PPM 512×512 affichant les arêtes d'un cube animé via LookAt et projection perspective ; (3) implémenter TRS complet et vérifier la décomposition inverse sur 20 triplets aléatoires.

4 Semaine 4 — Module 04 : Quaternions

Durée : 6 heures

4.1 Problème du gimbal lock

Les angles d'Euler (yaw, pitch, roll) souffrent du **gimbal lock** : lorsque le pitch atteint ±90, les axes yaw et roll deviennent coplanaires et le système perd un degré de liberté. De plus, l'interpolation LERP entre deux orientations Euler ne suit pas le chemin le plus court sur la sphère des rotations, ce qui produit des mouvements non-naturels.

4.2 Définition et structure

Un quaternion $q = w + xi + yj + zk$ avec $i^2 = j^2 = k^2 = ijk = -1$. Un **quaternion unitaire** ($\|q\| = 1$) représente une rotation de $2\arccos(w)$ radians autour de l'axe (x, y, z).

Opérations fondamentales :

- **Conjugué** : $q^* = (w, -x, -y, -z)$ — inverse la rotation.
- **Produit de Hamilton** : composition de rotations (non-commutatif).
- Rotation d'un vecteur : $v' = q \otimes (0, v) \otimes q^*$, optimisée sans construire deux quaternions intermédiaires.

Conversion axe-angle → quaternion :

$$q = \left(\cos\frac{\theta}{2},\ \hat{n}\sin\frac{\theta}{2}\right)$$

4.3 SLERP — interpolation sphérique

Le **SLERP** (*Spherical Linear intERPolation*) suit le chemin le plus court sur la sphère des quaternions unitaires avec une vitesse angulaire *constante* :

$$\mathrm{Slerp}(a, b, t) = \frac{\sin((1-t)\Omega)}{\sin\Omega}\,a + \frac{\sin(t\Omega)}{\sin\Omega}\,b, \qquad \Omega = \arccos(a \cdot b)$$

Lorsque Ω est très petit ($\cos\Omega > 0.9999$), on bascule vers un LERP normalisé pour éviter la division par zéro. Si $a \cdot b < 0$, on inverse b pour prendre le chemin court.

4.4 Méthode de Shepperd — conversion Quat ↔ Mat3

La conversion quaternion → matrice 3×3 est directe (9 produits). La conversion inverse (**méthode de Shepperd**) est numériquement stable car elle choisit, selon la trace de R, la composante dominante du quaternion pour éviter les divisions par de petites valeurs :

- `trace > 0` : calculer w en priorité.
- `R(0,0)` dominant : calculer x en priorité.
- Idem pour les cas y et z dominants.

4.5 Travaux pratiques

Les TP demandent : (1) valider les quaternions (rotation, aller-retour Quat/Mat3, inverse) ; (2) animer en 60 frames une rotation entre deux orientations et comparer visuellement SLERP (vitesse angulaire uniforme) et LERP (vitesse non-uniforme) dans le rasteriseur logiciel du Module 03.

4 Synthèse

Semaine	Module	Durée	Apport clé
1	IEEE 754 & flottants	6h	Précision numérique, Kahan, Welford
2	Vec2d, Vec3d, Vec4d	9h	Algèbre vectorielle, dot, cross, Gram-Schmidt
3	Mat3d, Mat4d	9h	Transformations, Rodrigues, LookAt, TRS
4	Quaternions	6h	Rotations stables, SLERP, Shepperd
Total S1 (4 semaines)		**30h**	Bibliothèque `NkMath` — fondations

Ces quatre modules forment le socle mathématique sur lequel reposent tous les algorithmes du pipeline AR : SVD (M05), détection de features (M07–08), calibration (M10–11), estimation de pose PnP (M15) et rendu OpenGL (M18–20).